KONNECT — Backend Architecture

1. Service Split

Service	Language	Responsibility
API/Orchestration service	JavaScript (Node.js)	Auth, user DB access, client-facing REST API, webhook receipt, job queueing, caching, retry/backoff logic, billing
Intelligence service	Python	Parsing, embeddings, incremental knowledge graph updates, incremental GNN re-embedding, PCST retrieval, LLM answer generation
Inter-service transport	Protobuf over gRPC	Typed contracts between JS and Python; supports streaming for incremental graph-update events

The Python service owns essentially all output the frontend renders (graph deltas, chat answers, citations). JS is a thin, fast layer in front of it.

---

2. JavaScript Backend (Orchestration Layer)

Concern	Choice	Why
Framework	**Express** (or Fastify if raw throughput matters later)	Matches synopsis; minimal overhead for a thin orchestration layer
Job queue	**BullMQ** on **Redis**	Debouncing rapid edits, idempotent retry handling, backed by Redis you're already running
Webhook handling	Express routes + BullMQ producers	Receive Drive/OneDrive push notifications → enqueue job, ack fast
Auth	**OAuth 2.0** (Google Drive, Microsoft Graph for OneDrive) + session/JWT for app users	Standard, both platforms support it natively
RPC client to Python	`@grpc/grpc-js` + `google-protobuf` / `ts-proto` for generated types	Typed, fast, avoids hand-synced JSON schemas
Semantic cache	Redis (vector-capable via **RediSearch/RedisVL** or a simple stored-embedding + manual cosine compare)	Cache past query→answer pairs; embed incoming query, cosine-match against cached queries before hitting the Python service

Concern	Choice	Why
Retry logic	Custom exponential backoff (base delay × 2^attempt, jitter, max 5 attempts) wrapping both webhook fetch calls and gRPC calls to the Python service	Matches your stated max-5-retry requirement; add jitter to avoid thundering-herd retries
Billing	**Stripe** (Checkout + Webhooks for subscription lifecycle events)	Standard for SaaS-style plan tiers
User DB	**PostgreSQL** via **Prisma** or **Drizzle ORM**	Type-safe queries, migrations, good DX with Node

Shared contract layer: put `.proto` files in a shared package (e.g. `/proto` at repo root, or its own tiny package) so both the JS and Python sides generate stubs from the same source of truth instead of hand-syncing types.

---

3. Python Intelligence Layer

Concern	Choice	Why
Framework	**FastAPI**	Async-first, plays well with gRPC servicer running alongside it, matches synopsis
gRPC server	`grpcio` + `grpcio-tools`	Receives parse/update/query requests from the JS layer
Text preprocessing	**spaCy** (not NLTK)	Faster runtime, better production-grade tokenization/NER pipeline, easier to swap in transformer-based pipelines later
Embeddings	**sentence-transformers** (e.g. a small/fast model like `all-MiniLM-L6-v2` for latency, or a larger model if quality matters more than speed)	Standard, fast, well-supported; swap-in point if you later want a hosted embedding API
Entity/relationship extraction	LLM-based extraction (per GraphRAG), wrapped in your own class	You're wrapping Microsoft's `graphrag` pipeline logic rather than calling it as an opaque batch job — lets you intercept per-document extraction for incremental use
GraphRAG integration	**Custom wrapper class inspired by/around Microsoft's `graphrag` SDK**	Use their SDK for the initial full-corpus bootstrap index; your own class handles the per-document incremental path, since their pipeline is batch-only

Concern	Choice	Why
Community detection	**Neo4j Graph Data Science (GDS) library** — has Leiden built-in	Avoids reimplementing Leiden; keeps it colocated with the graph store, scoped to local neighborhoods for incremental updates
Deep learning framework	**PyTorch**	Matches synopsis, ROLAND/G-Retriever reference implementations are PyTorch
GNN layers	**PyTorch Geometric (PyG)** for base message-passing (GAT/GraphSAGE-style layers)	Use library for the *standard* GNN forward pass — no need to hand-roll this
Incremental update rule (ROLAND GRU)	**From-scratch implementation** (`torch.nn.GRUCell` as the primitive, but the hierarchical state update logic, per-node state cache, and truncated-BPTT training loop written by you)	This is your project's core research contribution — should not be a black-box library call
Subgraph retrieval (PCST)	**From-scratch implementation, or wrap a PCST approximation library** (e.g. `pcst_fast`) if the timeline is tight	`pcst_fast` gives you a correct, fast approximation; you can still write the prize-assignment (cosine similarity scoring) logic yourselves so the "from scratch" framing holds where it matters most
LLM calls (extraction + answer generation)	Anthropic/OpenAI API client (whichever you're using)	Frozen LLM per your synopsis — no fine-tuning needed
Graph storage	**Neo4j** (via `neo4j` Python driver / `neo4j-graphrag` helpers)	Native k-hop traversal, GDS Leiden, matches your incremental-neighborhood-update design far better than recursive Postgres CTEs
Vector index for embeddings	**Neo4j vector index** (if staying single-store) or a dedicated vector DB (e.g. Qdrant) if you want it decoupled from graph storage	Keeps embedding similarity search close to the graph; avoids a second infra dependency unless you need it

---

4. Data Stores — Task-to-DBMS Mapping

Data	Store	Reasoning
User accounts, auth, billing/plan state	**PostgreSQL**	Relational, transactional, standard for user/billing data

Data	Store	Reasoning
Knowledge graph (entities, relationships, communities)	**Neo4j**	Native graph traversal, k-hop neighborhood queries, built-in Leiden via GDS — directly matches your incremental update design
Node/edge embeddings for similarity search	**Neo4j vector index** (colocated)	Keeps retrieval (PCST prize assignment) close to the graph without a second lookup hop
Query→answer semantic cache	**Redis**	Fast in-memory lookup, TTL support, already used for the job queue
Job queue / webhook event buffer	**Redis (via BullMQ)**	Debouncing, retries, idempotency keys

---

5. Reliability & Latency Considerations

- **Exponential backoff, max 5 retries**, applied to: webhook content fetches (Drive/OneDrive API calls) and any Python-service gRPC call that fails transiently. Include jitter.
- **Idempotency keys** derived from each webhook's change event ID, checked before enqueueing, so retried deliveries don't double-process.
- **Debouncing per document**: rapid successive edits to the same file collapse into one job, processed after a short quiet window.
- **Cache invalidation**: when a cached answer's cited subgraph nodes get updated by an incremental graph update, evict or flag that cache entry — don't rely on TTL alone, since staleness is the exact failure mode this project is designed to avoid.
- **spaCy over NLTK**: faster tokenization/NER at inference time, matters since preprocessing sits on the critical path of "edit → updated answer" latency you're benchmarking in the evaluation section.
- **Bounded-neighborhood re-embedding**: the GRU update pass only touches nodes within k hops of the changed document's entities — this is the latency property your whole evaluation section (Section 7) is built around, so the Python service should expose k as a tunable parameter for your benchmarking.

---

6. From-Scratch vs. Library — Summary

Component	Approach
ROLAND GRU hierarchical state update (Eq. 5–7)	**From scratch** — core contribution
GraphRAG entity/relationship extraction + incremental adaptation	**Own wrapper class**, inspired by/around Microsoft's SDK skeleton

Component	Approach
PCST prize assignment (cosine similarity scoring)	**From scratch**
PCST solver itself	Library (`pcst_fast`) unless time allows a custom approximation
Base GNN message passing (GAT/GraphSAGE layers)	Library (PyTorch Geometric)
Leiden community detection	Library (Neo4j GDS)
Text preprocessing/NER	Library (spaCy)
Embeddings	Library (sentence-transformers)